Spring Batch Migration Changes Document

1. Objective

This document outlines all code and architectural changes made in transitioning from the legacy Quartz-based batch processing design to the final Spring Batch implementation.

2. Architectural Flow Change

Legacy Flow: A raw Quartz Job triggered a standard Spring Service that sequentially queried, validated, and saved batches via loops. Exceptions were handled manually, and chunks were processed one-by-one inside a large synchronous block.

New Flow (Spring Batch): The Quartz Scheduler is kept strictly as a trigger. Execution is handed off to Spring Batch which utilizes a Reader, Processor, and Writer paradigm. This improves transaction management, chunk-based reliability, and standardizes error handling/reruns.

3. Code Level Changes

3.1. ICaseBatchService & CaseBatchService

• The original createBatchCase() method was marked @Deprecated.

• Added a new executeBatchJob() method returning a JobExecution to allow the Quartz job to directly launch the Spring Batch framework.

• The core data validation method (createCaUniverseTList) was refactored and detached from the transactional DB save logic so it could be strictly used for data validation inside the ItemProcessor.

3.2. Addition of Spring Batch Components

• BatchIdItemReader: Reads distinct Batch IDs from the Database dynamically and acts as the entrypoint for the Step stream. Refactored to drop execution context state indexing to resolve restart/skipping bugs.

• ValidatingBatchProcessor: Manages the business logic validation mappings using the legacy verification checks. Now wrapped in a try-catch block to correctly log failures and dispatch failure warning emails organically upon data exceptions.

• BatchProcessingResult: A data wrapper to route valid items directly to standard processors, and corrupted batches directly to Invalid routing destinations.

• CaseEntryBatchWriter & InvalidClaimsBatchWriter: Segmented database operations allowing valid chunks to commit efficiently to individual DB queues via iCaseBatchIndividualService, while invalid chunks cleanly isolate themselves via caUniverseBatchRepository and trigger Invalid data emails to admins. Exceptions here are correctly escalated (thrown) to flag Spring Batch statuses as FAILED.

3.3. Configuration Management

• CaseEntryBatchConfiguration: Ties the Reader, Processor, and routing Writers together into a definitive Spring Batch Step with customizable chunk bounds (e.g. chunk size = 5).

• application.yml modified to spring.batch.job.enabled: false to avoid accidental launch on startup.

3.4. Metadata & Database Dependencies

• Eliminated legacy circular dependencies by utilizing @Lazy Job loading.

• Removed redundant & outdated obsolete classes (BatchIdProcessor, BatchIdReader, and CaseBatchJobConfig).

4. Database Scripts & Verification Queries

To correctly support Spring Batch State tracking and verify data integrity before and after the batch logic, the following metadata schema and diagnostic SQL queries have been utilized:

4.1. Spring Batch Metadata Object Creation (DDL)

```
CREATE TABLE BATCH_JOB_INSTANCE (
  JOB_INSTANCE_ID BIGINT NOT NULL PRIMARY KEY,
  VERSION BIGINT NULL,
  JOB_NAME VARCHAR(100) NOT NULL,
  JOB_KEY VARCHAR(32) NOT NULL,
  CONSTRAINT JOB_INST_UN UNIQUE (JOB_NAME, JOB_KEY)
);

CREATE TABLE BATCH_JOB_EXECUTION (
  JOB_EXECUTION_ID BIGINT NOT NULL PRIMARY KEY,
  VERSION BIGINT NULL,
  JOB_INSTANCE_ID BIGINT NOT NULL,
  CREATE_TIME DATETIME NOT NULL,
  START_TIME DATETIME NULL,
  END_TIME DATETIME NULL,
  STATUS VARCHAR(10) NULL,
  EXIT_CODE VARCHAR(2500) NULL,
  EXIT_MESSAGE VARCHAR(2500) NULL,
  LAST_UPDATED DATETIME NULL,
  CONSTRAINT JOB_INST_EXEC_FK FOREIGN KEY (JOB_INSTANCE_ID)
    REFERENCES BATCH_JOB_INSTANCE(JOB_INSTANCE_ID)
);

CREATE TABLE BATCH_JOB_EXECUTION_PARAMS (
  JOB_EXECUTION_ID BIGINT NOT NULL,
  PARAMETER_NAME VARCHAR(100) NOT NULL,
  PARAMETER_TYPE VARCHAR(100) NOT NULL,
```

```
  PARAMETER_VALUE VARCHAR(2500) NULL,
  IDENTIFYING CHAR(1) NOT NULL,
  CONSTRAINT JOB_EXEC_PARAMS_FK FOREIGN KEY (JOB_EXECUTION_ID)
    REFERENCES BATCH_JOB_EXECUTION(JOB_EXECUTION_ID)
);

CREATE TABLE BATCH_STEP_EXECUTION (
  STEP_EXECUTION_ID BIGINT NOT NULL PRIMARY KEY,
  VERSION BIGINT NOT NULL,
  STEP_NAME VARCHAR(100) NOT NULL,
  JOB_EXECUTION_ID BIGINT NOT NULL,
  CREATE_TIME DATETIME NOT NULL,
  START_TIME DATETIME NULL,
  END_TIME DATETIME NULL,
  STATUS VARCHAR(10) NULL,
  COMMIT_COUNT BIGINT NULL,
  READ_COUNT BIGINT NULL,
  FILTER_COUNT BIGINT NULL,
  WRITE_COUNT BIGINT NULL,
  READ_SKIP_COUNT BIGINT NULL,
  WRITE_SKIP_COUNT BIGINT NULL,
  PROCESS_SKIP_COUNT BIGINT NULL,
  ROLLBACK_COUNT BIGINT NULL,
  EXIT_CODE VARCHAR(2500) NULL,
  EXIT_MESSAGE VARCHAR(2500) NULL,
  LAST_UPDATED DATETIME NULL,
  CONSTRAINT JOB_EXEC_STEP_FK FOREIGN KEY (JOB_EXECUTION_ID)
    REFERENCES BATCH_JOB_EXECUTION(JOB_EXECUTION_ID)
);

CREATE TABLE BATCH_STEP_EXECUTION_CONTEXT (
  STEP_EXECUTION_ID BIGINT NOT NULL PRIMARY KEY,
  SHORT_CONTEXT VARCHAR(2500) NOT NULL,
  SERIALIZED_CONTEXT VARCHAR(MAX) NULL,
  CONSTRAINT STEP_EXEC_CTX_FK FOREIGN KEY (STEP_EXECUTION_ID)
    REFERENCES BATCH_STEP_EXECUTION(STEP_EXECUTION_ID)
);

CREATE TABLE BATCH_JOB_EXECUTION_CONTEXT (
  JOB_EXECUTION_ID BIGINT NOT NULL PRIMARY KEY,
  SHORT_CONTEXT VARCHAR(2500) NOT NULL,
  SERIALIZED_CONTEXT VARCHAR(MAX) NULL,
  CONSTRAINT JOB_EXEC_CTX_FK FOREIGN KEY (JOB_EXECUTION_ID)
    REFERENCES BATCH_JOB_EXECUTION(JOB_EXECUTION_ID)
);

CREATE SEQUENCE BATCH_STEP_EXECUTION_SEQ START WITH 0 MINVALUE 0 NO CYCLE NO
CACHE;
CREATE SEQUENCE BATCH_JOB_EXECUTION_SEQ START WITH 0 MINVALUE 0 NO CYCLE NO
CACHE;
CREATE SEQUENCE BATCH_JOB_SEQ START WITH 0 MINVALUE 0 NO CYCLE NO CACHE;
```

4.2. Operational & Verification Queries

Used to inspect the state of intelligent keys, provider history, assignments, and structural components of a batch run:

```
-- Validate Case creation and assignments
SELECT * FROM CA_INTELLIGENT_KEY_T WHERE CASE_ID = 9669;
SELECT * FROM CA_GENERAL_T  WHERE CASE_ID = 9669;
SELECT * FROM CA_PROVIDER_T WHERE CASE_ID = 9669;
```

```
-- Validate Full-scale and historical associations
SELECT * FROM CA_INVST_FULLSCALE_T   WHERE CASE_ID = 9669;
SELECT * FROM CA_INVST_FS_HIST_T     WHERE CASE_ID = 9669;
SELECT * FROM CA_INVST_FS_STATUS_HIST_T WHERE CASE_ID = 9669;
SELECT * FROM CA_INVST_FS_ISSUE_T    WHERE CASE_ID = 9669;
SELECT * FROM CA_INVST_FS_ISSUE_HIST_T WHERE CASE_ID = 9669;
SELECT * FROM CA_INVST_FS_ASSIGN_T   WHERE CASE_ID = 9669;
SELECT * FROM CA_INVST_FS_ASSIGN_HIST_T WHERE CASE_ID = 9669;

-- Validate Claims processing counts
SELECT COUNT(*) FROM CA_CLAIM_LIST_T              WHERE CASE_ID = 9669;
SELECT COUNT(*) FROM CA_CLAIM_CORRECTIONS_T       WHERE CASE_ID = 9669;
SELECT COUNT(*) FROM CA_CLAIM_CORRECTIONS_HIST_T  WHERE CASE_ID = 9669;

-- Universe target rows & queue deletion checks
SELECT COUNT(*) FROM CA_UNIVERSE_T WHERE CT_BATCH_ID = 452;
SELECT COUNT(*) FROM CA_UNIVERSE_BATCH_T WHERE CT_BATCH_ID = 452;

-- Validate Metadata Job Exection
SELECT TOP 10 * FROM BATCH_JOB_INSTANCE ORDER BY JOB_INSTANCE_ID DESC;
SELECT TOP 10 * FROM BATCH_JOB_EXECUTION ORDER BY JOB_EXECUTION_ID DESC;
SELECT TOP 10 * FROM BATCH_STEP_EXECUTION ORDER BY STEP_EXECUTION_ID DESC;

-- Email Audit Delivery
SELECT * FROM CA_EMAILS_BATCH_T WHERE CT_BATCH_ID = 451 ORDER BY EMAIL_DT DESC;
```